

Multi-Method Forecasting Lab - Walmart Store 1

Session 6 - Supply Chain Analytics (MGT-353)

Method

The dataset contains store-level weekly sales. Store 1 was selected. The final 12 weeks (10 Aug 2012 to 26 Oct 2012) were held out before fitting, so no test sales were used to fit any method.

Method	MAPE
Seasonal Naive (52-week + Holiday_Flag match)	3.52%
4-Week Simple Moving Average	3.79%
SES alpha = 0.3	3.89%
SES alpha = 0.6	3.91%

Reflection

1. What did you ask AI?

I asked AI to build three forecasting approaches for one Walmart store using a 12-week holdout: a 4-week simple moving average, simple exponential smoothing with alpha 0.3 and 0.6, and a seasonal-naive method that uses the holiday flag. I also asked it to calculate MAPE and compare the forecasts.

2. What does the Python output show?

The output shows that the seasonal-naive method had the lowest MAPE at 3.52%. The 4-week moving average was 3.79%, SES alpha 0.3 was 3.89%, and SES alpha 0.6 was 3.91%.

3. Supply-chain insight

Store 1 shows a recurring seasonal pattern. In the training data, holiday weeks averaged about 7.7% higher sales than non-holiday weeks. The test period contained only 1 holiday week, so most of the 12-week comparison reflects ordinary weekly demand rather than peak holiday demand.

4. Managerial decision

For this test period, I would use the seasonal-naive forecast for replenishment planning because it produced the lowest holdout MAPE (3.52%). I would still monitor a simple moving average as a benchmark because its error was also low and it reacts to recent demand.

5. Limitation

The holdout contains only 12 weeks and only one holiday week, so it is too short to fully test the benefit of holiday seasonality. Also, the dataset is store-level total sales rather than product/SKU-level demand, so the result cannot directly determine replenishment for individual items.

Google Colab Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("Walmart_Store_sales.csv")
df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y")

STORE_ID = 1
store_df = df[df["Store"] == STORE_ID].sort_values("Date").reset_index(drop=True)

train_df = store_df.iloc[:-12].copy()
test_df = store_df.iloc[-12:].copy()

# 1) 4-week Simple Moving Average
sma_forecast = np.repeat(train_df["Weekly_Sales"].tail(4).mean(), len(test_df))

# 2) Simple Exponential Smoothing, coded directly
def ses_level(series, alpha):
    level = series.iloc[0]
    for y in series.iloc[1:]:
        level = alpha*y + (1-alpha)*level
    return level

ses03_forecast = np.repeat(ses_level(train_df["Weekly_Sales"], 0.3), len(test_df))
ses06_forecast = np.repeat(ses_level(train_df["Weekly_Sales"], 0.6), len(test_df))

# 3) Seasonal naive with Holiday_Flag
# For each test week, look 52 weeks back and use the closest TRAINING week
# with the same Holiday_Flag. No test sales are used for fitting.
seasonal_forecast = []
for _, row in test_df.iterrows():
    target_date = row["Date"] - pd.Timedelta(weeks=52)
    candidates = train_df[train_df["Holiday_Flag"] == row["Holiday_Flag"]].copy()
    nearest_idx = (candidates["Date"] - target_date).abs().idxmin()
    seasonal_forecast.append(train_df.loc[nearest_idx, "Weekly_Sales"])
seasonal_forecast = np.array(seasonal_forecast)

def mape(actual, forecast):
    return np.mean(np.abs((actual - forecast) / actual)) * 100

actual = test_df["Weekly_Sales"].to_numpy()
comparison = pd.DataFrame({
    "Method": ["4-week SMA", "SES alpha=0.3", "SES alpha=0.6", "Seasonal Naive"],
    "MAPE (%)": [
        mape(actual, sma_forecast),
        mape(actual, ses03_forecast),
        mape(actual, ses06_forecast),
        mape(actual, seasonal_forecast)
    ]
}).sort_values("MAPE (%)")

print(comparison.to_string(index=False))

plt.figure(figsize=(11,6))
plt.plot(test_df["Date"], actual, marker="o", label="Actual")
plt.plot(test_df["Date"], sma_forecast, marker="o", label="4-week SMA")
plt.plot(test_df["Date"], ses03_forecast, marker="o", label="SES 0.3")
plt.plot(test_df["Date"], ses06_forecast, marker="o", label="SES 0.6")
plt.plot(test_df["Date"], seasonal_forecast, marker="o", label="Seasonal Naive")
plt.title(f"Walmart Store {STORE_ID}: 12-Week Forecast Comparison")
plt.xlabel("Date")
```

```
plt.ylabel("Weekly Sales")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```